

Giorno 10 — JPA CRUD e basi REST

Esercizio 1 — CRUD completo con JPA

Estendendo il progetto dell'esercizio precedente (o creandone uno nuovo), implementerai tutte le operazioni CRUD (Create, Read, Update, Delete) usando Spring Data JPA.

Cosa fare:

- Aggiorna l'entità `Studente` aggiungendo: `data_nascita` (`LocalDate`), `corso_laurea` (`String`). Aggiungi anche l'annotazione `@Column(nullable=false)` su nome e cognome. Riavvia: Hibernate deve aggiornare la tabella.
- Crea uno `StudenteDAO` (o usa direttamente il `Repository`). Implementa un metodo `findAll()` che restituisce tutti gli studenti, e un `findById(int id)` che restituisce uno studente per ID.
- Implementa un metodo `save(Studente s)` per salvare un nuovo studente, e un metodo `update(Studente s)` per aggiornare uno esistente. Usa `@Transactional` dove necessario.
- Implementa un metodo `deleteById(int id)` che elimina uno studente per ID.
- Crea una query JPQL personalizzata: `findByCorsoLaurea(String corso)` che restituisce tutti gli studenti iscritti a un dato corso. Usa `@Query` nel `Repository`.
- Usa il `CommandLineRunner` per testare TUTTE le operazioni CRUD: crea 3 studenti, leggili tutti, aggiorna il corso di laurea del primo, elimina il terzo, poi fai una query per corso. Stampa i risultati su console.
- Aggiungi la generazione automatica della tabella: metti `spring.jpa.hibernate.ddl-auto=create-drop` e verifica che ad ogni riavvio la tabella venga ricreata.

Consegna: La console dell'applicazione deve mostrare (grazie a `show-sql=true`) tutte le query SQL generate da Hibernate durante il test CRUD. Copia le query in un file `hibernate-queries.txt`.

Esercizio 2 — Il tuo primo REST Controller

Crea i tuoi primi endpoint REST per esporre i dati degli studenti. Introduci il concetto di risposta JSON, metodi HTTP e codici di stato.

Cosa fare:

- Crea un `@RestController` `StudenteController` con il mapping base `/api/studenti`.
- Implementa un endpoint `GET /api/studenti` che restituisce la lista di tutti gli studenti. Testalo nel browser o con Postman.
- Implementa un endpoint `GET /api/studenti/{id}` che restituisce un singolo studente per ID. Se lo studente non esiste, restituisci un `ResponseEntity` con codice 404.
- Implementa un endpoint `POST /api/studenti` che accetta nel body un oggetto JSON con i dati di un nuovo studente (`@RequestBody`), lo salva e restituisce lo studente salvato con codice 201 (`Created`).
- Implementa un endpoint `PUT /api/studenti/{id}` che aggiorna i dati di uno studente esistente. Se l'ID non esiste, risponde con 404.
- Implementa un endpoint `DELETE /api/studenti/{id}` che elimina uno studente. Risponde con 204 (`No Content`) se l'eliminazione ha successo, 404 se non trovato.
- Testa tutti gli endpoint con Postman e documenta le chiamate nel file `README.md` del progetto.

Consegna: Tutti e 5 gli endpoint devono funzionare correttamente. Crea una collection Postman con tutte le chiamate di test. Esporta e pusha la collection